TP3 : Sécurité des Applications Web
2026-2027

Authentification par jetons JWT et gestion des identités avec Keycloak

> **Informations pratiques :**
> **Durée** : 3 séances de TP (les présentations de groupe auront lieu lors de la quatrième séance).
> **Groups** : travail en groupe de 3 élèves maximum.
> Chaque élève du groupe doit être capable de :
> - Comprendre l'ensemble des parties du projet ;
> - Répondre aux questions portant sur toutes les étapes et toutes les implémentations réalisées.
>
> **Utilisation de l'IA** : une utilisation limitée de l'IA est autorisée, à condition que chaque utilisation soit correctement citée.
> **Éléments attendus** : un compte rendu et une présentation devant la classe d'environ 12 minutes.
> Structure du compte rendu : le compte rendu suit le plan ci-dessous. Toutes les captures d'écran doivent être numérotées, légendées et commentées dans le texte.
> - Page de garde
> - Introduction (contexte, versions des outils)
> - Partie 1 – JWT : architecture, génération du jeton, tests des routes, cookie HttpOnly et CSRF
> - Partie 2 – Keycloak : introduction à Keycloak, configuration, tests Postman, analyse du jeton
> - Conclusion
> - Annexes (répartition des tâches)

Objectifs

Ce TP est composé de deux parties :

- **Première partie – JWT :** créer une application Express/Node.js qui met en œuvre une authentification basée sur des jetons **JWT** (*JSON Web Token*), et tester cette application avec Postman.
- **Deuxième partie – Keycloak :** déployer le serveur d'autorisation **Keycloak** dans un conteneur Docker, le configurer (realm, client, rôles, utilisateurs) et tester les flux OAuth2 / OpenID Connect avec Postman.

Sources :

Support de cours IUT Belfort-Montbéliard (TP10 – côté serveur).

L. Gupta, « Getting Started with Keycloak on Docker », HowToDoInJava, 17 avril 2023 : https://howtodoinjava.com/devops/keycloak-on-docker/

Site officiel de Keycloak : https://www.keycloak.org/

Première partie : authentification par jetons JWT

1. Rappels : l'authentification basée sur des jetons

Par rapport à l'authentification par cookie de session, où l'état de la session est conservé côté serveur, le jeton **JSON Web Token (JWT)** est stocké **côté client** : stockage local pour le navigateur, *SharedPreferences* pour Android. Le serveur n'a donc pas besoin de mémoriser la session.

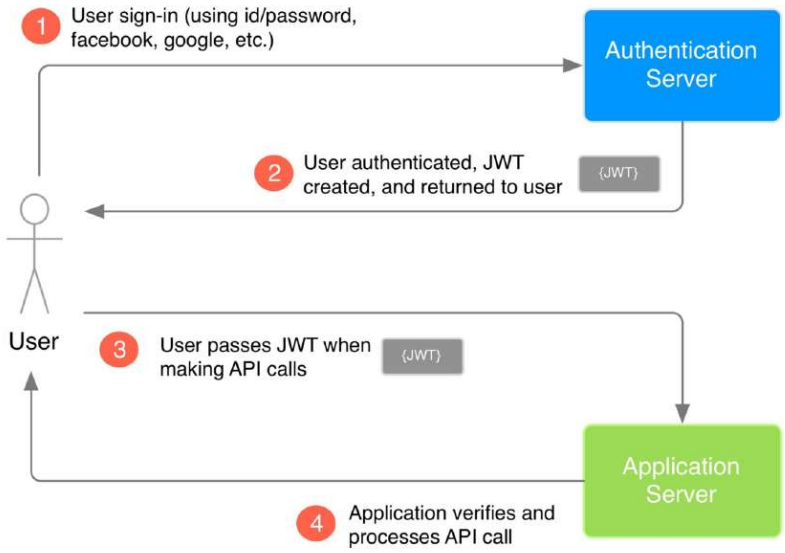


Figure 1 : *Principe de l'authentification par jeton : le serveur d'authentification émet un JWT que l'utilisateur présente à chaque appel d'API.*

Un JWT signé numériquement comporte trois parties : **Header**, **Payload** et **Signature**. Elles sont combinées selon la structure standard `header.payload.signature`.

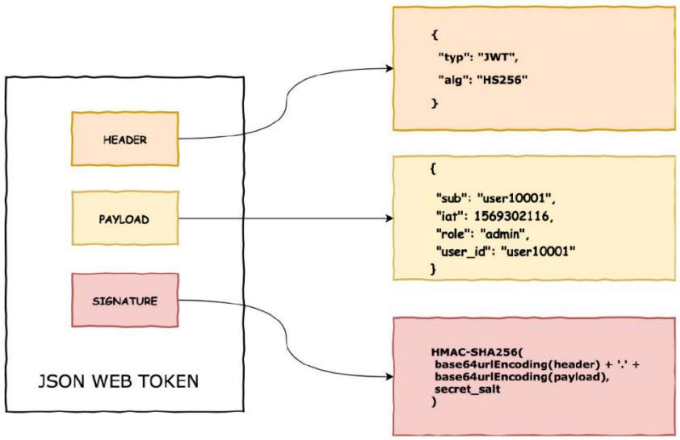


Figure 2 : *Structure d'un JSON Web Token (en-tête, charge utile, signature).*

Le client attache généralement le JWT dans l'en-tête d'autorisation (`Authorization`) avec le préfixe `Bearer` :

```
Authorization: Bearer <jeton>
```

ou dans l'en-tête `x-access-token` :

```
x-access-token: <jeton>
```

2. API à fournir

Méthode	URL	Action
POST	`/users/signin`	Connecter un compte utilisateur et renvoyer un jeton.
GET	`/users/getList`	Récupérer du contenu protégé (liste des utilisateurs).
GET	`/verifyToken`	Vérifier la validité d'un jeton.
GET	`/users/logout`	Déconnecter l'utilisateur et supprimer les cookies et les jetons d'accès.

3. Structure du code

- **Un middleware d'authentification JWT.** Cette couche vérifie le jeton. Si le middleware génère une erreur, un message est envoyé en réponse HTTP.

```
const authMiddleware = function (req, res, next) {
  // 1. recuperer le jeton (en-tete Authorization ou cookie)
  // 2. verifier sa signature et sa date d'expiration
  // 3. en cas d'erreur : res.status(401).json({ message: "..." })
  // 4. sinon : next()
};
```

- **Les différentes routes :**

```
// Authentifie l'utilisateur et genere un jeton
app.post('/users/signin', function (req, res) { /* ... */ });

// Affiche la liste des utilisateurs (route protegee)
app.get('/users/getList', authMiddleware, (req, res) => { /* ... */ });
```

> **Note**
> Pour générer le jeton, vous utiliserez les modules `jsonwebtoken` et `moment` :

```
const jwt = require('jsonwebtoken');
const moment = require('moment');
```

4. Qu'est-ce qu'un middleware Node.js ?

> **Définition (documentation Express)**
> Les fonctions de middleware sont des **fonctions** qui peuvent accéder à l'objet requête (`req`), à l'objet réponse (`res`) et à la fonction middleware suivante dans le cycle requête-réponse de l'application. La fonction middleware suivante est couramment désignée par une variable nommée `next`.
>
> *Source : https://expressjs.com/fr/guide/using-middleware.html*

Un middleware est exécuté avant que chaque requête ne soit routée vers le bon endpoint. Il se situe entre la requête et la réponse :



Figure 3 : *Position du middleware dans le cycle requête-réponse.*

Créez votre premier middleware

Supposons que l'on veuille enregistrer le chemin de chaque requête. Sans middleware, on pourrait le faire avec un code comme celui-ci (fichier `TestMiddleware.js`, serveur sur le port 8080) :

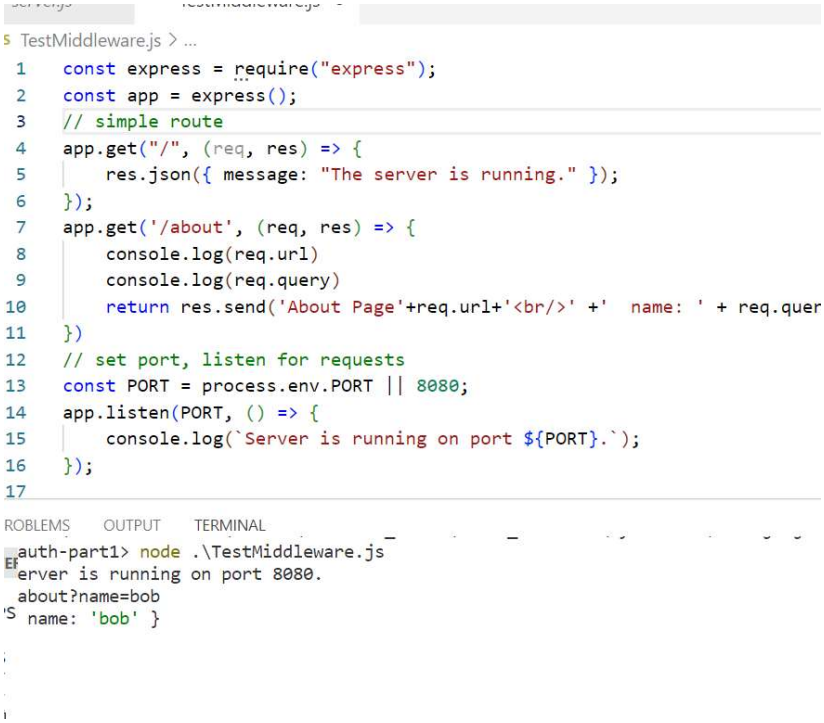


Figure 4 : *Journalisation écrite directement dans la route `/about` (code et sortie console).*

Le résultat dans le navigateur, pour la requête `http://localhost:8080/about?name=bob` :

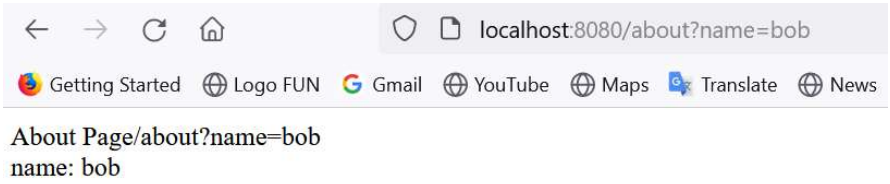


Figure 5 : *Réponse de la route `/about` dans le navigateur.*

Cela fonctionne très bien, mais que se passe-t-il avec 100 chemins différents ? On ne peut pas copier-coller ces lignes `console.log()` dans toutes les routes. **Une solution consiste à placer cette journalisation dans une fonction middleware :**

```
TestMiddleware.js > ...
1   const express = require("express");
2   const app = express();
3
4   const logger  = (req, res, next) => {
5       console.log(req.url)
6       console.log(req.query)
7       next()
8   }
9   app.use(logger) // execute your middleware for all requests
10  app.get('/about', (req, res) => {
11      return res.send('About Page'+req.url+'<br/>' +'  name: ' + req.query
12  })
13  // set port, listen for requests
14  const PORT = process.env.PORT || 8080;
15  app.listen(PORT, () => {
16      console.log(`Server is running on port ${PORT}.`);
17  });
18
```

```
PROBLEMS   OUTPUT   TERMINAL
auth-part1> node .\TestMiddleware.js
Server is running on port 8080.
/about?name=bob
{ name: 'bob' }
```

Figure 6 : *Journalisation déplacée dans un middleware `logger` déclaré avec `app.use()`.*

Lorsque l'utilisateur visite `/about`, Express exécute le middleware spécifié dans `app.use()`. Une fois le middleware exécuté, `next()` poursuit l'exécution vers `app.get()`. Séquence d'exécution :

requête client → /about ⇒ logger() → app.get() → réponse client

Lors de la création d'un middleware, il n'est **pas obligatoire** d'appeler `next()`. Le middleware peut envoyer sa propre réponse et remplacer, voire omettre complètement, `app.get()` :

```
JS TestMiddleware.js > app.get('/about') callback
1   const express = require("express");
2   const app = express();
3
4   const logger  = (req, res, next) => {
5       console.log(req.url)
6       console.log(req.query)
7       return res.send('About Page'+req.url+'<br/>' +'  name: ' + req.query.name)
8   }
9   app.use(logger) // execute your middleware for all requests
10  app.get('/about', (req, res) => {
11
12  })
13  // set port, listen for requests
14  const PORT = process.env.PORT || 8080;
15  app.listen(PORT, () => {
16      console.log(`Server is running on port ${PORT}.`);
17  });
18
```

```
PROBLEMS   OUTPUT   TERMINAL
TERMINAL
t1> node .\TestMiddleware.js
Server is running on port 8080.
/about?name=bob
{ name: 'bob' }
```

Figure 7 : *Middleware qui répond directement, sans appeler `next()` : la route `app.get()` n'est pas exécutée.*

À retenir :

- Un middleware peut être exécuté uniquement pour les requêtes situées sous un chemin donné : `app.use('/api', logger)`.
- Plusieurs middlewares peuvent être utilisés en même temps : `app.use([logger, auth])`.
- Un middleware peut modifier l'objet `req` et l'objet `res`.

5. Travail demandé – Partie 1 : JWT

Implémenter une authentification basée sur des jetons JWT et tester l'implémentation avec Postman (ou un outil similaire).

Lors de l'authentification d'un utilisateur, par exemple **Alice**, un JWT est généré par le serveur ; les informations contenues dans le jeton sont signées à l'aide d'une **clé secrète**, puis le jeton est transmis au client. Le jeton est ensuite renvoyé avec chaque requête HTTP, ce qui permet au serveur d'identifier l'utilisateur.

Étape 1 : **Préparer le projet.** Créer un dossier de projet, puis initialiser et installer les dépendances :

```
npm init -y
npm install express jsonwebtoken moment cookie-parser
```

Avant de passer à l'authentification, reproduisez les trois expériences de la section 4 (`TestMiddleware.js` : journalisation dans la route, middleware `logger`, middleware sans `next()`) pour vous familiariser avec le fonctionnement des middlewares. L'application JWT écoutera sur le **port 4000**.

Étape 2 : **Implémenter la route `POST /users/signin`.** Prévoir au moins deux utilisateurs (par exemple Alice avec le mot de passe `p1`, et Bob). Si les identifiants sont corrects, générer un JWT signé avec une clé secrète et une durée de validité limitée (calculée avec `moment`), puis renvoyer au client l'utilisateur, le jeton et sa date d'expiration.

Avec Postman, envoyer une requête **POST** vers `http://localhost:4000/users/signin` avec un corps JSON contenant le nom d'utilisateur et le mot de passe d'Alice :

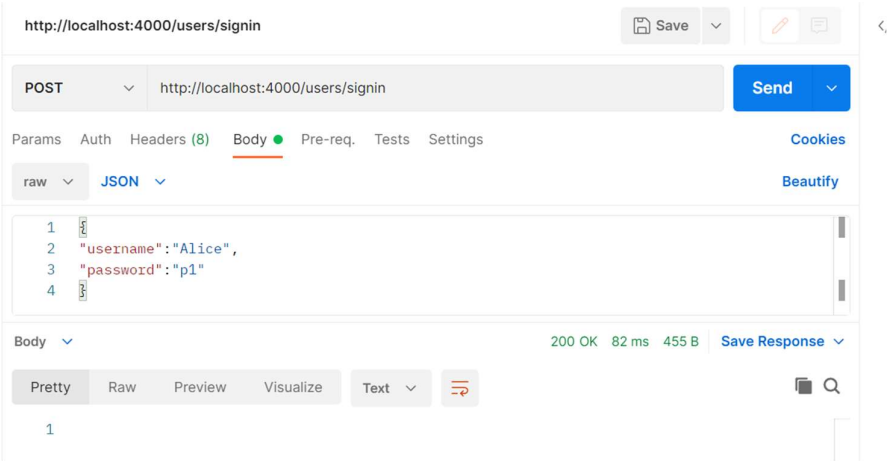


Figure 8 : *Requête POST /users/signin envoyée avec Postman (nom d'utilisateur et mot de passe d'Alice).*

Le serveur renvoie le jeton :

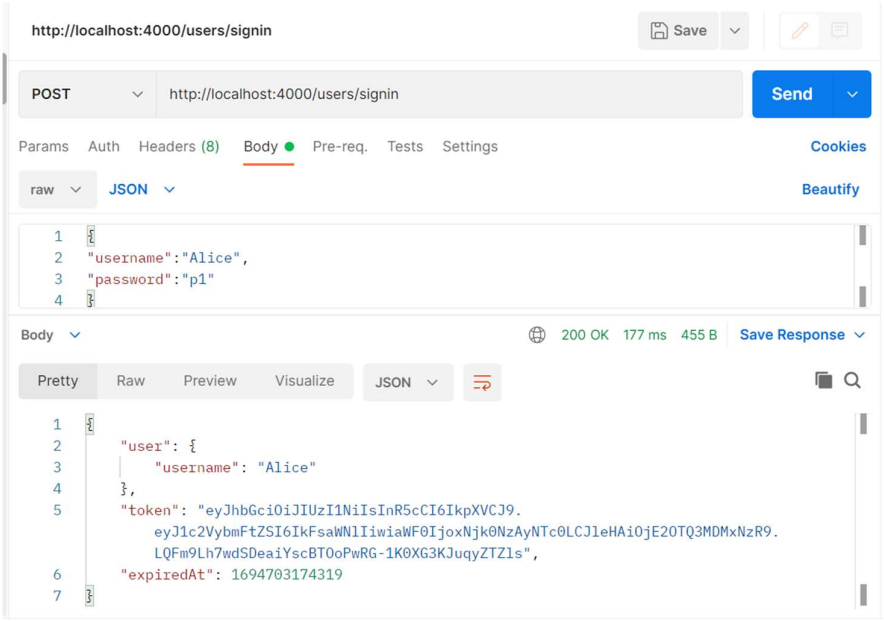


Figure 9 : *Réponse du serveur : utilisateur, jeton JWT et date d'expiration.*

Étape 3 : **Écrire le middleware `authMiddleware` et protéger `GET /users/getList`.** Le middleware lit le jeton dans l'en-tête `Authorization: Bearer <jeton>`, vérifie sa signature et sa date d'expiration, puis appelle `next()` ; sinon il répond avec une erreur (`401`).

Le jeton obtenu peut alors être utilisé pour afficher la liste des utilisateurs. Dans Postman, ajouter l'en-tête `Authorization` avec le préfixe `Bearer` :

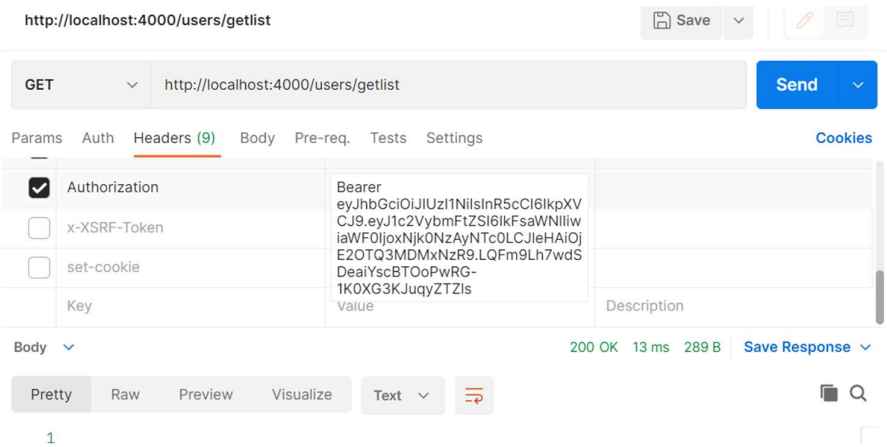


Figure 10 : *Requête GET /users/getList avec le jeton dans l'en-tête Authorization.*

Réponse attendue :

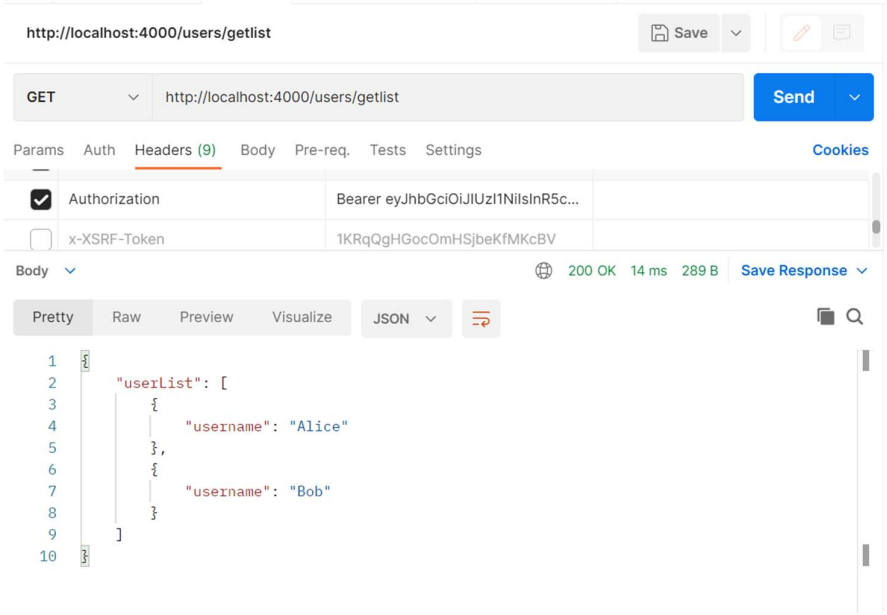


Figure 11 : *Liste des utilisateurs renvoyée par la route protégée.*

Vérifier également qu'une requête **sans jeton**, avec un jeton **invalide** ou **expiré**, est rejetée.

Étape 4 : **Implémenter `GET /verifyToken` et `GET /users/logout`.** La première route indique si le jeton présenté est valide ; la seconde supprime le cookie et le jeton d'accès côté client.

Étape 5 : **Question de réflexion – où stocker son jeton JWT ?** Deux choix s'offrent à vous (source : simplx.fr) :

- un **cookie** ;
- le **Web Storage** de HTML5 (`localStorage` ou `sessionStorage`).

> **Web Storage ou cookie ?**
>
> Le Web Storage est accessible par tout code JavaScript du même domaine : il est donc sensible aux attaques par ***cross-site scripting*** **(XSS)**, c'est-à-dire l'injection de JavaScript malveillant dans votre page web.
>
> Les cookies sont protégés des attaques XSS grâce à l'attribut **`HttpOnly`**, qui empêche l'accès via le JavaScript du domaine, mais ils sont sensibles à un autre type d'attaque : la ***Cross-Site Request Forgery*** **(CSRF)**.
>
> *Source : https://website.simplx.fr/blog/2016/09/27/authentification-api-via-jwt-et-cookies/*

Étape 6 : **Stocker le JWT dans un cookie `HttpOnly`.** Modifier votre application pour que le jeton ne soit plus renvoyé au client dans le corps de la réponse, mais placé dans un cookie `HttpOnly` (avec les attributs `Secure` et `SameSite`). Pour vous protéger contre les attaques CSRF, ajoutez un **jeton anti-CSRF** :

- lors du `signin`, générer une valeur aléatoire (`xsrfToken`), l'inclure dans le payload du JWT et la renvoyer au client dans la réponse ;
- le client renvoie cette valeur dans l'en-tête `x-xsrf-token` à chaque requête ;
- `authMiddleware` lit désormais le JWT dans le cookie (`cookie-parser`) et vérifie que l'en-tête `x-xsrf-token` correspond à la valeur contenue dans le jeton ;

– `/users/logout` supprime le cookie.

Tester avec Postman : la requête vers `/users/getList` transmet maintenant le cookie contenant le JWT ainsi que l'en-tête `x-xsrf-token`.

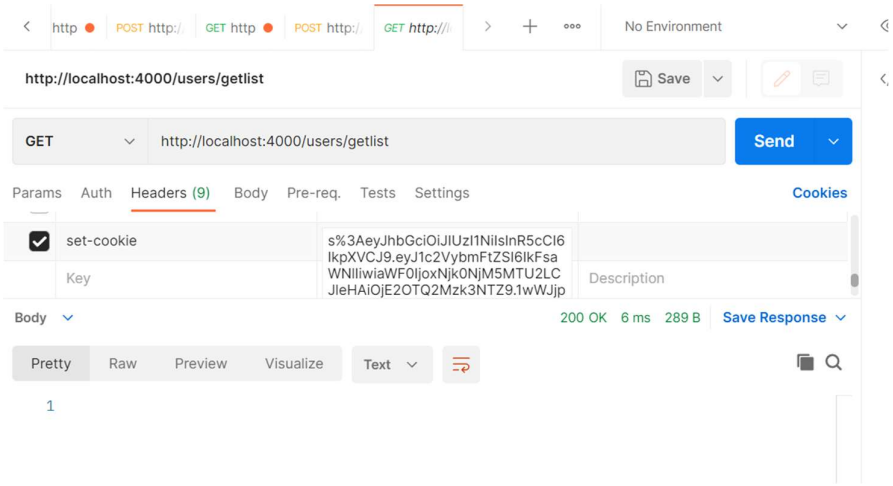


Figure 12 : *Test de /users/getList avec le jeton transmis dans un cookie (et l'en-tête x-xsrf-token).*

Référence : « Login App with CSRF protection – Understanding authentication using JWT access token and refresh token », Clue Mediator (consulté le 14 sept. 2023) : https://www.cluemediator.com/login-app-with-csrf-protection-understanding-authentication-using-jwt-access-token-and-refresh-token

Deuxième partie : Keycloak

Le contenu de cette partie est adapté de : L. Gupta, « Getting Started with Keycloak on Docker », HowToDoInJava, 17 avril 2023 (consulté le 17 septembre 2023). Autres références : https://www.keycloak.org/

1. Objectifs

Découvrir le serveur d'autorisation Keycloak exécuté dans un conteneur Docker, ses différentes configurations (realm, client, rôles, utilisateurs) et tester le flux d'obtention d'un jeton d'autorisation avec Postman.

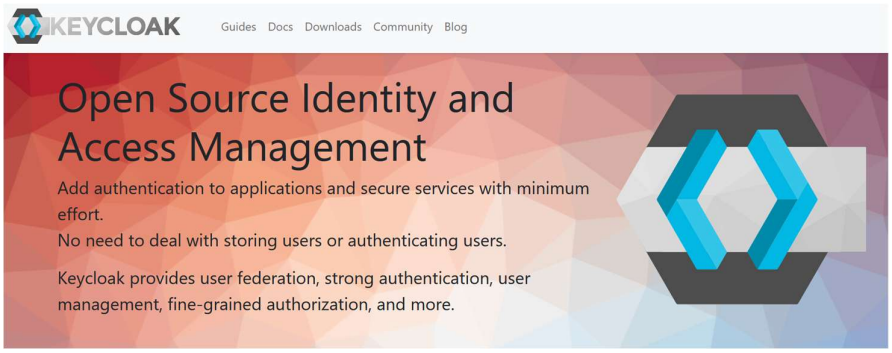


Figure 13 : *Keycloak, solution open source de gestion des identités et des accès (https://www.keycloak.org).*

Keycloak est un système open source de **gestion des identités et des accès** (*Identity and Access Management*, IAM). C'est un serveur compatible **OAuth2, OpenID Connect et SAML** qui fournit différents services d'authentification et d'autorisation, tels que l'authentification unique (*single sign-on*) ou la connexion via les réseaux sociaux.

2. OAuth2 avec Keycloak

Keycloak joue le rôle de **serveur d'autorisation** OAuth2 : il émet des jetons d'accès et les vérifie lors des requêtes suivantes.

3. Terminologie

- **Client :** application qui demande à Keycloak d'authentifier et d'autoriser ses utilisateurs.
- **Utilisateur (*User*) :** entité pouvant se connecter à l'application cliente. Il possède des attributs (e-mail, nom d'utilisateur, adresse, téléphone, date de naissance), peut appartenir à des groupes et se voir attribuer des rôles.
- **Rôle :** identifie un type ou une catégorie d'utilisateurs. Administrateur, utilisateur, manager ou employé sont des rôles typiques.
- **Groupe :** permet de gérer des ensembles d'utilisateurs. Des attributs et des rôles peuvent être associés à un groupe ; les membres d'un groupe héritent de ces attributs et de ces correspondances de rôles.
- **Realm :** gère un ensemble d'utilisateurs, d'identifiants, de rôles et de groupes. Un utilisateur appartient à un realm et s'y connecte. Les realms sont isolés les uns des autres et ne peuvent gérer et authentifier que les utilisateurs qu'ils contrôlent.

4. Installation et démarrage du serveur

Étape 1 : **Installer Docker** sur votre machine si ce n'est pas déjà fait.

Étape 2 : **Récupérer l'image Keycloak** depuis Docker Hub :

```
docker pull quay.io/keycloak/keycloak:22.0.3
```

> **Remarque sur la version**
> Les captures d'écran de ce TP ont été réalisées avec **Keycloak 22.0.3**. Nous fixons cette version pour que l'interface corresponde aux figures. Avec `latest`, l'interface peut différer et, à partir de Keycloak 26, les variables d'administration `KEYCLOAK_ADMIN` / `KEYCLOAK_ADMIN_PASSWORD` sont remplacées par `KC_BOOTSTRAP_ADMIN_USERNAME` / `KC_BOOTSTRAP_ADMIN_PASSWORD`.

Étape 3 : **Choisir le mode de démarrage.** Le serveur peut être démarré selon deux modes :

- **Mode développement :** offre des fonctionnalités facilitant le développement et les tests. Il est destiné uniquement aux environnements de développement et ne doit **pas** être utilisé en production.
- **Mode production :** offre de meilleures performances et des fonctionnalités de sécurité renforcées.

Dans ce TP, nous utiliserons Keycloak en mode développement.

Étape 4 : **Démarrer le conteneur.** La commande suivante démarre un conteneur nommé `keycloak_dev`, expose le port **8180** (le port par défaut est 8080) sur la machine hôte, et définit l'identifiant et le mot de passe de l'administrateur à `admin` :

```
docker run --name keycloak_dev -p 8180:8180 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:22.0.3 start-dev --http-port=8180
```

Vérifier dans la console que le serveur a bien démarré (message indiquant que Keycloak écoute sur le port 8180) :

```
C:\coursesENSIBS24\websec23\keycloak>docker run --name keycloak_srv -p 8180:8180  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_AD
MIN_PASSWORD=admin quay.io/keycloak/keycloak:latest start-dev --http-port=8180
Updating the configuration and installing your custom providers, if any. Please wait.
2023-09-17 11:40:16,149 INFO  [io.quarkus.deployment.QuarkusAugmentor] (main) Quarkus augmentation completed in 8283ms
2023-09-17 11:40:17,601 INFO  [org.keycloak.quarkus.runtime.hostname.DefaultHostnameProvider] (main) Hostname settings:
Base URL: <unset>, Hostname: <request>, Strict HTTPS: false, Path: <request>, Strict BackChannel: false, Admin URL: <uns
et>, Admin: <request>, Port: -1, Proxied: false
2023-09-17 11:40:19,365 WARN  [io.quarkus.agroal.runtime.DataSources] (main) Datasource <default> enables XA but transac
tion recovery is not enabled. Please enable transaction recovery by setting quarkus.transaction-manager.enable-recovery=
true, otherwise data may be lost if the application is terminated abruptly
2023-09-17 11:40:20,012 WARN  [org.infinispan.PERSISTENCE] (keycloak-cache-init) ISPN000554: jboss-marshalling is deprec
ated and planned for removal
2023-09-17 11:40:20,188 WARN  [org.infinispan.CONFIG] (keycloak-cache-init) ISPN000569: Unable to persist Infinispan int
ernal caches as no global state enabled
2023-09-17 11:40:20,287 INFO  [org.infinispan.CONTAINER] (keycloak-cache-init) ISPN000556: Starting user marshaller 'org
.infinispan.jboss.marshalling.core.JBossUserMarshaller'
2023-09-17 11:40:20,990 INFO  [org.keycloak.connections.infinispan.DefaultInfinispanConnectionProviderFactory] (main) No
de name: node_466119, Site name: null
2023-09-17 11:40:22,040 INFO  [org.keycloak.quarkus.runtime.storage.legacy.liquibase.QuarkusJpaUpdaterProvider] (main) I
nitializing database schema. Using changelog META-INF/jpa-changelog-master.xml
2023-09-17 11:40:24,374 INFO  [org.keycloak.broker.provider.AbstractIdentityProviderMapper] (main) Registering class org
.keycloak.broker.provider.mappersync.ConfigSyncEventListener
2023-09-17 11:40:24,417 INFO  [org.keycloak.services] (main) KC-SERVICES0050: Initializing master realm
2023-09-17 11:40:26,614 INFO  [io.quarkus] (main) Keycloak 22.0.3 on JVM (powered by Quarkus 3.2.5.Final) started in 10.
280s. Listening on: http://0.0.0.0:8180
2023-09-17 11:40:26,614 INFO  [io.quarkus] (main) Profile dev activated.
2023-09-17 11:40:26,615 INFO  [io.quarkus] (main) Installed features: [agroal, cdi, hibernate-orm, jdbc-h2, jdbc-mariadb
```

Figure 14 : *Journal de démarrage du conteneur Keycloak en mode développement.*

Étape 5 : **Ouvrir la console d'administration.** Le serveur écoutant sur le port 8180, ouvrir l'URL `http://localhost:8180/` dans un navigateur.

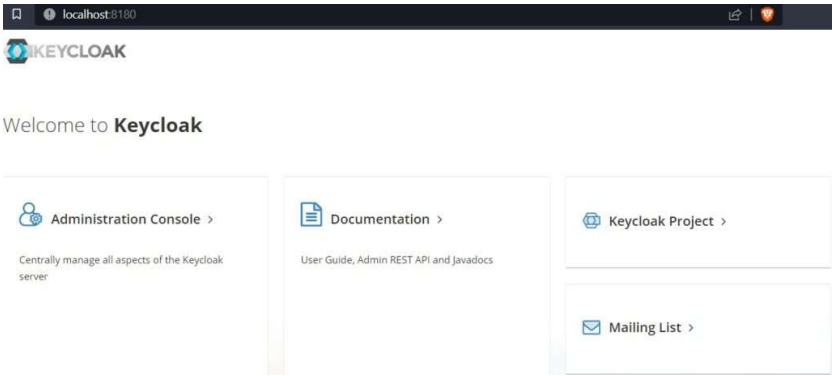


Figure 15 : *Page d'accueil de Keycloak.*

Cliquer sur le lien **Administration Console** pour afficher la page de connexion, saisir `admin` / `admin`, puis cliquer sur **Sign In**.

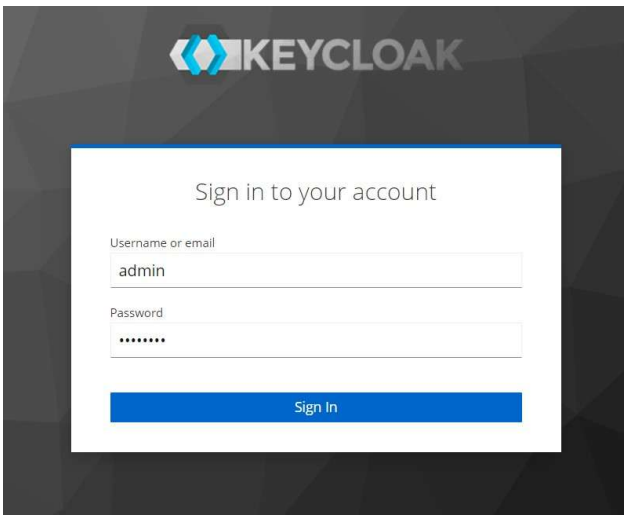


Figure 16 : *Page de connexion à la console d'administration.*

5. Configuration du serveur

Étape 1 : **Créer un nouveau realm.** Un realm gère un ensemble d'utilisateurs, d'identifiants, de rôles et de groupes ; il est isolé des autres realms. Dans la liste déroulante en haut à gauche (realm *master*), cliquer sur **Create Realm**, saisir le nom `realmtest`, laisser l'option **Enabled** à *On* puis cliquer sur **Create**.

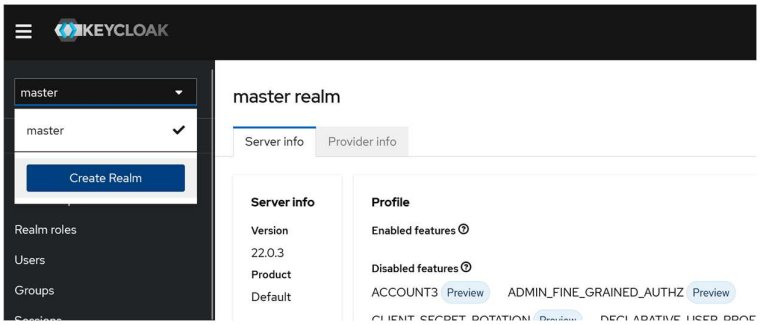


Figure 17 : *Menu de sélection du realm et bouton Create Realm.*

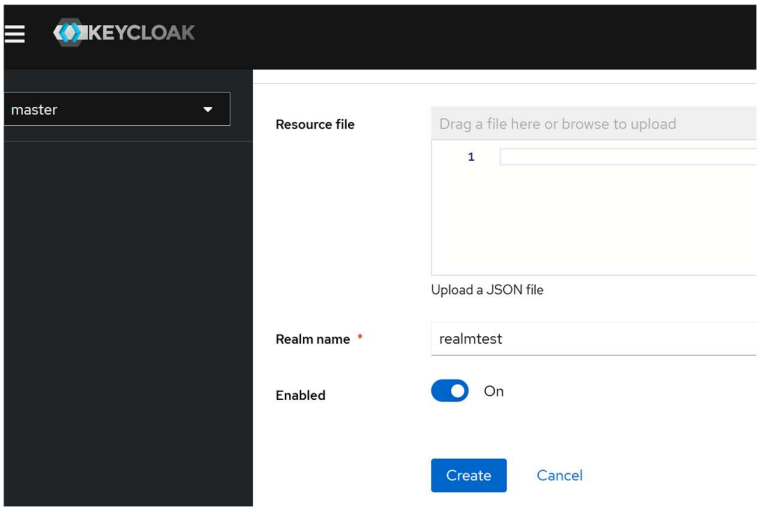


Figure 18 : *Création du realm `realmtest`.*

Étape 2 : **Ajouter un client.** Les clients sont le plus souvent des applications ou des services qui demandent à Keycloak d'authentifier un utilisateur. Dans le realm `realmtest`, ouvrir le menu **Clients** dans le panneau de gauche, puis cliquer sur **Create client**.

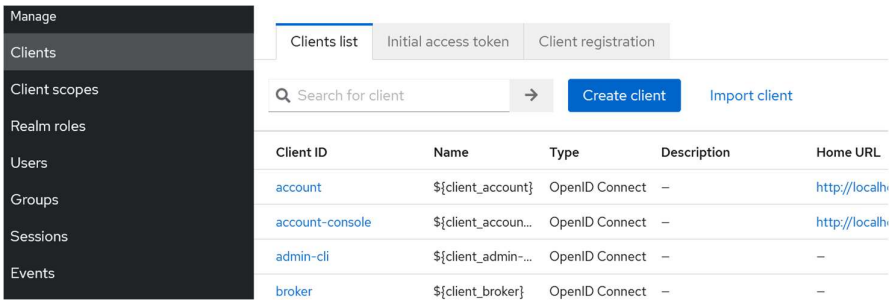


Figure 19 : *Liste des clients du realm.*

Renseigner le *Client ID* (`employee-management-api`) et le nom (`Employee Management API`), puis cliquer sur **Next**.

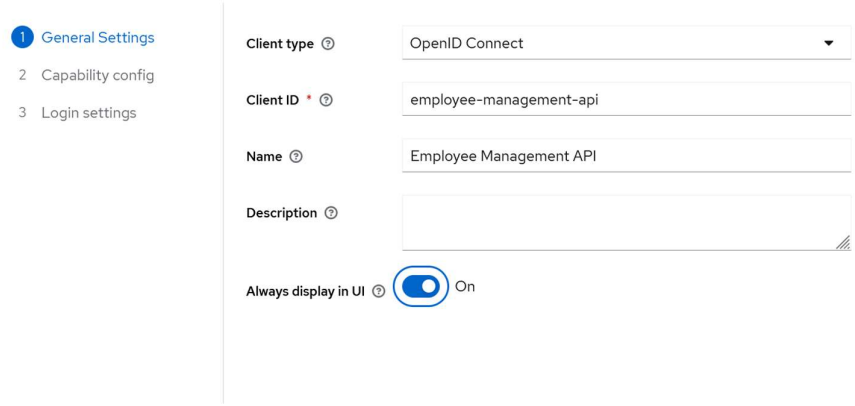


Figure 20 : *Paramètres généraux du client.*

Choisir les capacités appropriées (les infobulles donnent de l'aide sur chaque option). Activer **Client authentication** : c'est ce qui fournit un *Client secret* dans l'onglet **Credentials**, nécessaire plus loin.

- **Standard flow :** active l'authentification OpenID Connect standard par redirection avec code d'autorisation (*Authorization Code Flow*).
- **Direct access grants :** le client a accès au nom d'utilisateur et au mot de passe et les échange directement contre un jeton d'accès auprès de Keycloak (*Resource Owner Password Credentials Grant*).

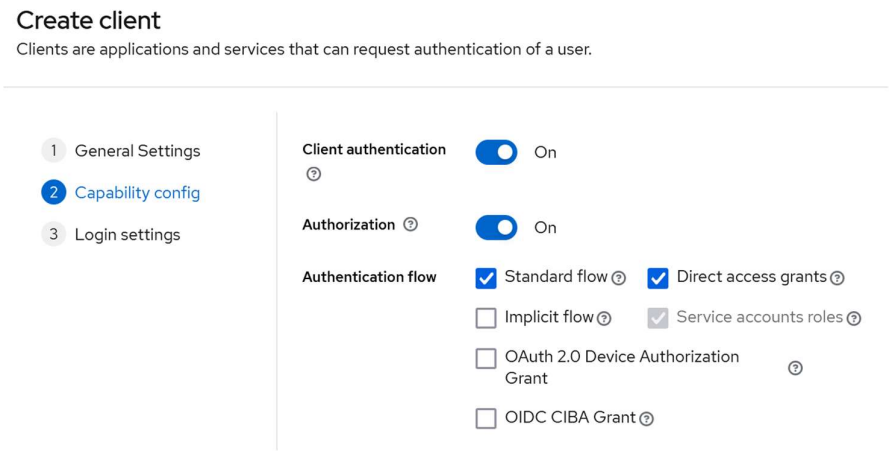


Figure 21 : *Configuration des capacités du client (Standard flow et Direct access grants).*

Renseigner ensuite l'URL racine de l'application cliente, les URL de redirection, etc. D'autres réglages sont disponibles dans les autres onglets.

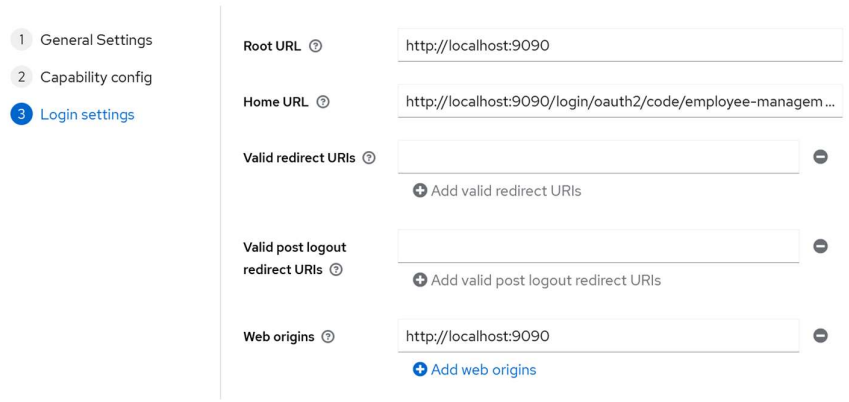


Figure 22 : *Paramètres de connexion du client (Root URL, Home URL, Web origins).*

Noter le **Client secret** de l'onglet **Credentials** : il servira aux clients pour demander le jeton d'accès.

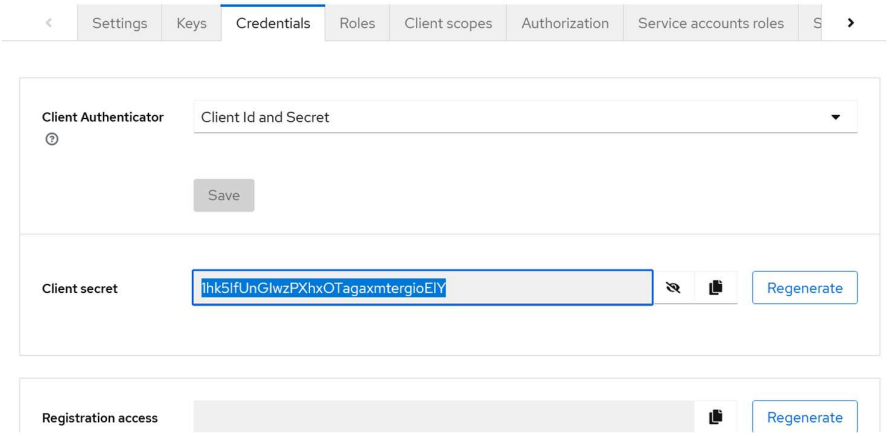


Figure 23 : *Onglet Credentials : Client secret du client.*

Étape 3 : **Créer des rôles.** Ajouter des rôles personnalisés utilisés pour l'autorisation : créer les deux rôles `USER` et `ADMIN` (menu **Realm roles** puis **Create role**).

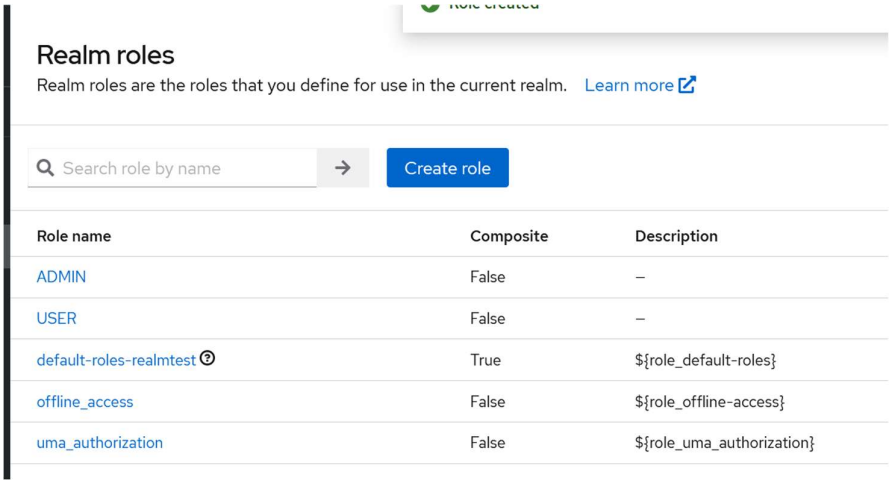


Figure 24 : *Rôles USER et ADMIN créés dans le realm.*

Étape 4 : **Créer un utilisateur.** Cliquer sur **Users** dans le panneau de gauche, puis sur **Add user**, et renseigner les informations (par exemple nom d'utilisateur `test-user` et e-mail `test-user@test.com`).

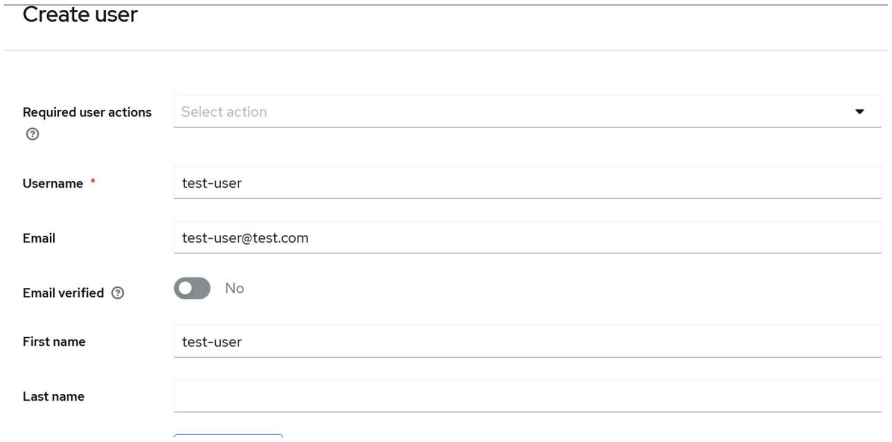


Figure 25 : *Formulaire de création d'un utilisateur.*

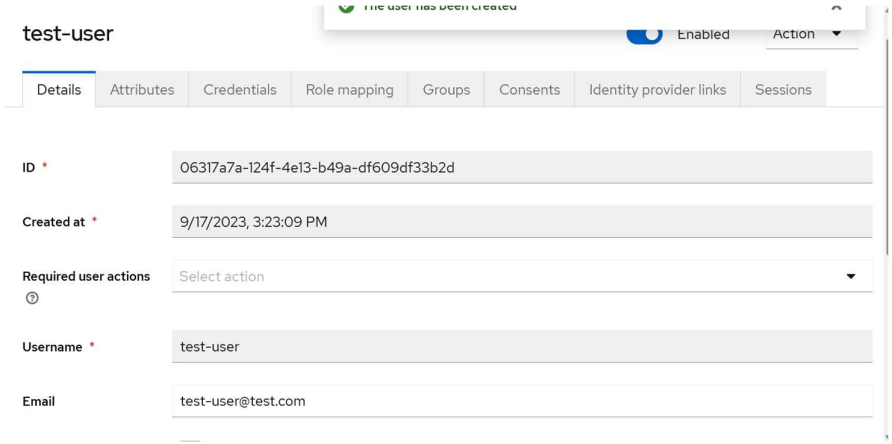


Figure 26 : *Détails de l'utilisateur `test-user` après création.*

Comme nous utilisons une authentification par nom d'utilisateur / mot de passe, définir le mot de passe dans l'onglet **Credentials** (bouton **Set password**).

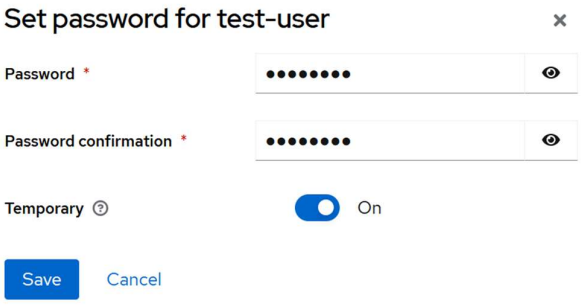


Figure 27 : *Définition du mot de passe de l'utilisateur.*

Attention

Sur la figure, l'option **Temporary** est à *On*. Il faut la passer à **Off** : sinon Keycloak demandera de changer le mot de passe à la première connexion et l'obtention du jeton avec Postman échouera.

À cette étape, il est possible d'associer un rôle à l'utilisateur dans l'onglet **Role mapping**. Dans ce TP, conservez les paramètres par défaut.

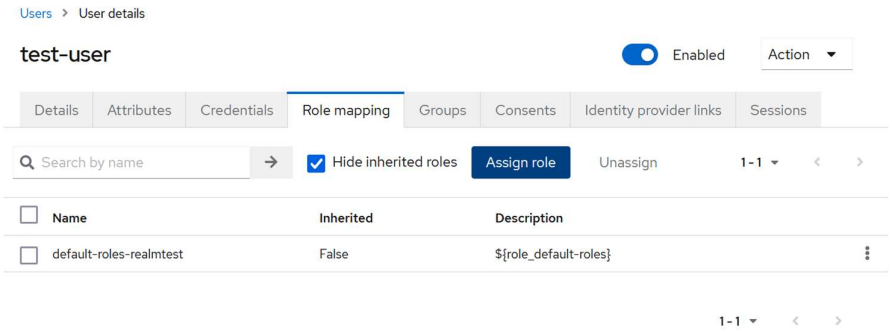


Figure 28 : *Onglet Role mapping de l'utilisateur (rôle par défaut).*

6. Tests avec Postman

Testons les différentes configurations, l'authentification et l'autorisation avec Postman.

Étape 1 : **Consulter la configuration OpenID.** Vérifier la configuration OpenID Connect (OIDC) à l'adresse `{{server}}/realms/{{realm}}/.well-known/openid-configuration`, c'est-à-dire ici `http://localhost:8180/realms/realmtest/.well-known/openid-configuration`. Cette configuration contient des informations sur la manière dont un client peut authentifier et autoriser des utilisateurs avec OIDC : URL de l'émetteur (*issuer*), points d'accès *authorization*, *token* et *userinfo*, types de réponses, types de sujets, *claims* et *scopes* pris en charge.

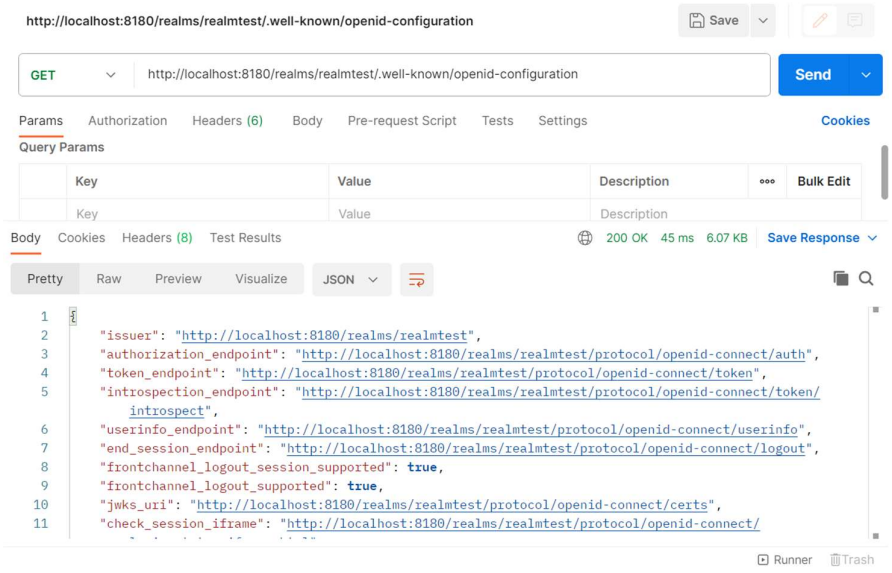


Figure 29 : *Réponse de la configuration OpenID Connect du realm `realmtest`.*

Étape 2 : **Demander un jeton d'authentification.** Envoyer une requête HTTP **POST** à l'URL du point d'accès *token* indiquée dans la configuration OIDC :

`http://localhost:8180/realms/realmtest/protocol/openid-connect/token`. Le corps de la requête est de type `x-www-form-urlencoded` avec les paramètres suivants :

Paramètre	Valeur
`username`	nom de l'utilisateur créé (`test-user`)
`password`	mot de passe défini pour cet utilisateur
`client_id`	`employee-management-api`
`client_secret`	secret noté dans l'onglet **Credentials** du client
`grant_type`	`password`
`scope`	`openid`

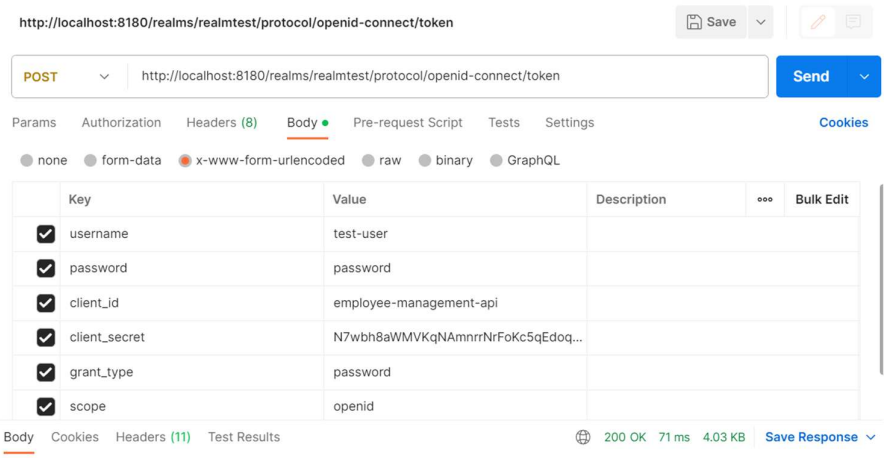


Figure 30 : *Requête POST vers le point d'accès token de Keycloak.*

En réponse, on obtient la structure suivante :

```
{
  "access_token":
"eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJwUERMNGc5S3Jp...",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": "...",
  "token_type": "Bearer",
  "id_token": "...",
  "not-before-policy": 0,
  "session_state": "b2c2c19e-5f0d-46a7-a917-8e04714ecf3b",
  "scope": "openid profile email"
}
```

Étape 3 : **Accéder aux informations de l'utilisateur.** Les informations de l'utilisateur constituent une ressource protégée : il faut transmettre le jeton d'accès dans l'en-tête `Authorization` (type *Bearer Token*). Envoyer une requête **GET** vers le point d'accès `/userinfo` indiqué dans la

configuration OIDC : `http://localhost:8180/realms/realmtest/protocol/openid-connect/userinfo`.

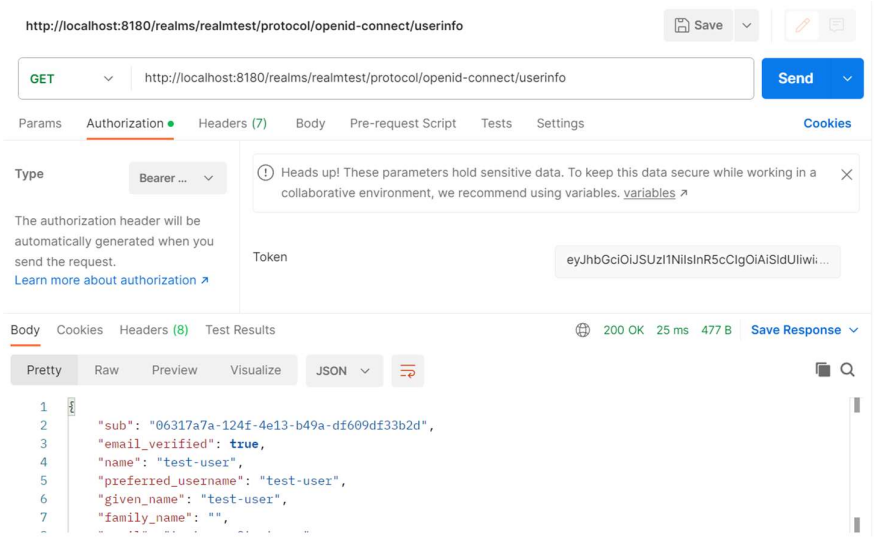


Figure 31 : *Requête GET /userinfo avec le jeton d'accès et informations de l'utilisateur renvoyées.*

Étape 4 : **Analyser le jeton.** Décoder le `access_token` (par exemple avec https://jwt.io) et comparer sa structure (header, payload, signature) avec celle du jeton que votre application Express génère dans la partie 1. Quels sont les champs présents (`iss`, `exp`, `realm_access`, etc.) ? Quel algorithme de signature est utilisé ?

Étape 5 : **Expiration du jeton.** Attendre plus de 300 secondes (`expires_in`), puis rappeler `/userinfo` avec le même jeton. Que constatez-vous ? À quoi sert le `refresh_token` ?

Pour aller plus loin

Le flux *Direct access grants* (grant_type=password) est pratique dans le cadre d'un TP, mais il est **déconseillé en production** : l'application cliente manipule directement le mot de passe de l'utilisateur, et ce flux a été retiré du projet de spécification OAuth 2.1. Le flux recommandé est le ***Standard flow*** (code d'autorisation), idéalement associé à **PKCE**.

Deux pistes d'approfondissement :

- **Utiliser le *Standard flow*** : configurer le client Keycloak et obtenir le jeton via le code d'autorisation, au lieu du flux *Direct access grants*.
- **Valider les jetons Keycloak dans votre application Express** : adapter l'application pour qu'elle vérifie les jetons émis par Keycloak, au lieu de ceux qu'elle génère elle-même. La signature se vérifie avec la clé publique exposée à l'adresse jwks_uri de la configuration OIDC.